VHGRAD

Development of a Didactic Deep Learning Framework from Scratch

Virvi Huta

2026

Contents

Technical Progression

1. Foundations of Neural Computation
Fundamental concepts underlying modern neural networks.

2. Building the First Components
Single neurons, multi-neuron layers, tensors and matrix operations with NumPy.

3. Structured Layer Design
Encapsulation of dense layers into reusable classes with organized data handling.

4. Non-Linearity through Activation
Step, linear, sigmoid, ReLU and Softmax activation functions and their roles.

5. Measuring Network Error
Cross-entropy loss formulation and accuracy metrics for classification.

6. Parameter Optimization
First principles of gradient-based learning and iterative parameter updates.

7. Calculus of Learning
Numerical and analytical derivatives, partial derivatives and the chain rule.

8. Gradient Flow through the Network
Full backward pass with analytically derived gradients across layers and losses.

9. Advanced Optimization Methods
SGD, learning rate scheduling, momentum and adaptive optimizers including Adam.

10. Controlling Generalization
Weight regularization, dropout and validation strategies to reduce overfitting.

11. Extending to Other Task Types
Binary classification and regression with appropriate outputs and loss functions.

12. Training on Real Data
End-to-end pipeline covering loading, preprocessing, batching and model training.

13. Deployment and Reuse
Parameter serialization, model loading and inference on unseen data.

1. Foundations of Neural Computation

1.1 The Single Neuron

A neuron is the fundamental computational unit of a neural network. It takes a set of inputs, multiplies each by a corresponding weight, sums the results and adds a scalar offset called a bias. Given inputs $x_1, x_2, \ldots, x_n$, weights $w_1, w_2, \ldots, w_n$ and bias b, the output of a single neuron is:

$$z = \sum_{i=1}^{n} x_i w_i + b \tag{1.1}$$

The weights control how strongly each input influences the output. A large positive weight amplifies the corresponding input, a negative weight inverts it and a weight near zero makes that input nearly irrelevant. The bias shifts the output independently of the inputs.

The first implementation computed this explicitly for four inputs with no abstraction:

```
inputs  = [1, 2, 3, 2.5]
weights = [0.2, 0.8, -0.5, 1.0]
bias    = 2

output = (inputs[0] * weights[0] +
          inputs[1] * weights[1] +
          inputs[2] * weights[2] +
          inputs[3] * weights[3] +
          bias)
# output: 4.8
```

Substituting the values directly: $1 \times 0.2 + 2 \times 0.8 + 3 \times (-0.5) + 2.5 \times 1.0 + 2 = 4.8$

Writing every multiplication by hand was intentional — the goal was to see the raw mechanics before introducing any abstraction.

1.2 A Layer of Neurons

A layer is a collection of neurons that all receive the same input vector but each apply their own weights and bias. For a layer of m neurons with shared input $\mathbf{x} \in \mathbb{R}^n$, the output of neuron j is:

$$z_j = \sum_{i=1}^{n} x_i w_{ji} + b_j, \quad j = 1, \ldots, m \tag{1.2}$$

The implementation first wrote all three outputs explicitly — every multiplication typed out — to make the repetitive structure visible. Then it was refactored into a loop:

```
inputs  = [1, 2, 3, 2.5]
weights = [[0.2, 0.8, -0.5, 1.0],
           [0.5, -0.91, 0.26, -0.5],
           [-0.26, -0.27, 0.17, 0.87]]
biases  = [2, 3, 0.5]

layer_outputs = []
for neuron_weights, neuron_bias in zip(weights, biases):
    neuron_output = 0
    for n_input, weight in zip(inputs, neuron_weights):
        neuron_output += n_input * weight
    neuron_output += neuron_bias
    layer_outputs.append(neuron_output)

# layer_outputs: [4.8, 1.21, 2.385]
```

The loop makes clear that a layer is nothing more than the same neuron computation repeated across rows of the weight matrix. The structure — same inputs, different weights per neuron — is the core of what a dense layer is.

1.3 The Dot Product

Before moving to NumPy it was important to recognise that a neuron's weighted sum is mathematically a dot product. Given two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, their dot product is:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i \tag{1.3}$$

This was verified manually and then with a loop for $\mathbf{a} = [1, 2, 3]$ and $\mathbf{b} = [2, 3, 4]$:

$$1 \times 2 + 2 \times 3 + 3 \times 4 = 20 \tag{1.4}$$

Recognising this equivalence is what makes the transition to NumPy clean — a neuron's computation is exactly one call to a dot product function.

1.4 Vectorized Computation with NumPy

Writing loops in Python is slow. NumPy replaces them with highly optimised C routines that operate on entire arrays at once. The single neuron collapses to one line:

```
outputs = np.dot(inputs, weights) + bias
# outputs: 4.8
```

For the full layer the weight matrix is passed first so NumPy computes one dot product per row — one output per neuron:

```
outputs = np.dot(weights, inputs) + biases
# outputs: [4.8, 1.21, 2.385]
```

The argument order matters. `np.dot(weights, inputs)` treats each row of the weight matrix as one neuron's weight vector and computes its dot product with the input vector, producing one scalar per neuron.

1.5 Matrices and Shape Alignment

A plain Python list passed to NumPy produces a 1D array — a vector with no orientation. Wrapping it in an extra list creates a 2D row vector of shape $(1, n)$, and transposing it with `.T` produces a column vector of shape $(n, 1)$. This was verified with a matrix dot product:

```
a = np.array([[1, 2, 3]])      # shape (1, 3)
b = np.array([[2, 3, 4]]).T    # shape (3, 1)
np.dot(a, b)                   # [[20]], shape (1, 1)
```

The result is a $(1, 1)$ matrix rather than a scalar because matrix multiplication was performed rather than a vector dot product. Understanding how shapes interact under multiplication is essential for everything that follows.

1.6 Batched Forward Pass

Processing one sample at a time is inefficient. In practice a neural network processes a batch of samples simultaneously. If the input matrix X has shape (N, n) — N samples each with n features — and the weight matrix W has shape (m, n) — m neurons each with n weights — then the full layer output is:

$$Z = XW^T + \mathbf{b} \tag{1.5}$$

where $\mathbf{b} \in \mathbb{R}^m$ is the bias vector broadcast across all samples, and Z has shape (N, m) — one output per neuron per sample.

The transpose W^T flips W from (m, n) to (n, m) so that the matrix product XW^T of shapes $(N, n) \cdot (n, m)$ is defined and yields (N, m).

```
inputs  = [[1.0, 2.0, 3.0, 2.5],
           [2.0, 5.0, -1.0, 2.0],
           [-1.5, 2.7, 3.3, -0.8]]

weights = [[0.2, 0.8, -0.5, 1.0],
           [0.5, -0.91, 0.26, -0.5],
           [-0.26, -0.27, 0.17, 0.87]]

biases = [2.0, 3.0, 0.5]

layer_outputs = np.dot(inputs, np.array(weights).T) + biases
```

The result is a $(3, 3)$ matrix — three samples, three neuron outputs each:

$$Z = \begin{pmatrix} 4.800 & 1.210 & 2.385 \\ 8.900 & -1.810 & 0.200 \\ 1.410 & 1.051 & 0.026 \end{pmatrix} \tag{1.6}$$

Each row corresponds to one input sample and each column to one neuron. This is a complete batched forward pass through a single dense layer — the foundation on

which every subsequent component of VHgrad is built.